



How Spark Works Internally: RDD and DAG

Dano Lee

Why RDD?

What was the problem in Hadoop/MapReduce?

- ❖ Every MapReduce job writes intermediate data to disk between stages (Map → Shuffle → Reduce).
- ❖ This I/O-heavy model causes:
 - ❖ High latency due to disk read/write.
 - ❖ No data reuse across jobs: you must recompute or reload from HDFS every time.
 - ❖ Poor support for iterative algorithms (e.g., ML, graph processing).

The Redundancy Problem in MapReduce

- ❖ In a machine learning algorithm like k-means clustering, you often iterate over the same data 10~100 times.
- ❖ In MapReduce, each iteration is a new job — reading the same data from HDFS again, doing computation, and writing intermediate output back to disk.
- ❖ This results in heavy I/O bottlenecks.

No Support for Data Reuse in MapReduce

- ❖ Say you pre-process a dataset (e.g., tokenize and clean up text).
- ❖ You want to use this clean version for both a frequency count and a sentiment analysis task.
- ❖ In MapReduce, you would have to redo the preprocessing in both jobs or manually save it to disk and reload, adding latency and complexity.

Motivation for RDDs

- ❖ **Overcoming Redundancy and Lack of In-Memory Reuse in MapReduce**

What did Spark do differently?

- ❖ Introduced **RDDs (Resilient Distributed Datasets)** — immutable, distributed collections of objects that can be stored in memory across stages.
- ❖ RDDs support:
 - ❖ **In-memory caching and reuse.**
 - ❖ **Fault tolerance via lineage** instead of replication.
- ❖ **Key Benefit of RDDs:**
 - ❖ Reduced I/O overhead and better performance for iterative and interactive workloads, which MapReduce handled poorly.

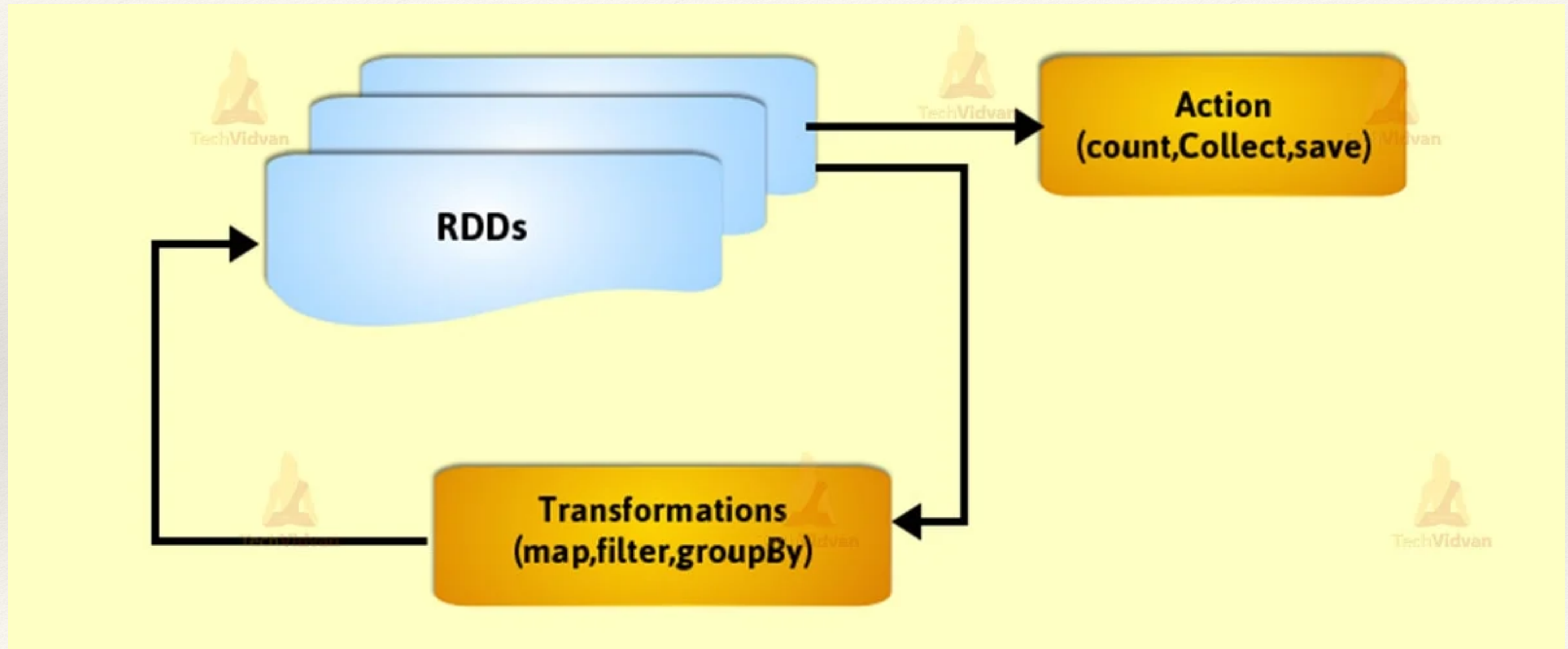
RDD

- ❖ **Resilient Distributed Dataset**
 - ❖ It is the **fundamental unit of data** in Apache Spark. RDDs are distributed a collection of elements across cluster nodes.
 - ❖ One of the Important parameters is **RDD supports parallel operations.**
 - ❖ Spark RDDs are **immutable in nature**. We can not make any changes though it can generate by transforming existing RDD.

Creating RDD

- ❖ We can create RDD in Spark by several ways.
 1. Parallelized collections
 2. External datasets
 3. Existing RDDs

Transformation vs Action



Lazy Evaluation

- ❖ Lazy evaluation in Spark means that **transformations** on RDDs are not executed immediately.
- ❖ Instead, Spark keeps track of the operations applied to each RDD (its **lineage**) and waits until an **action** is called to execute them.
- ❖ This allows Spark to **optimize the execution plan and minimize unnecessary computations**.

Transformation vs Action

❖ Transformations

- ❖ Transformations **produce new RDDs from the data in existing ones** using operations
- ❖ operations that are lazily evaluated and return a new RDD, DataFrame, or Dataset.
- ❖ Examples include **map, filter, flatMap, groupBy**, etc.
- ❖ Transformations are not executed immediately; they are only executed when an action is called.

❖ Actions

- ❖ An action **does not create a new RDD**, but returns the result of a computation on the data set
- ❖ operations that **trigger the evaluation of transformations** and return a result to the driver program or write data to external storage.
- ❖ Examples include **collect, count, saveAsTextFile, reduce**, etc.

Narrow & Wide Transformations

❖ **Narrow Transformations**

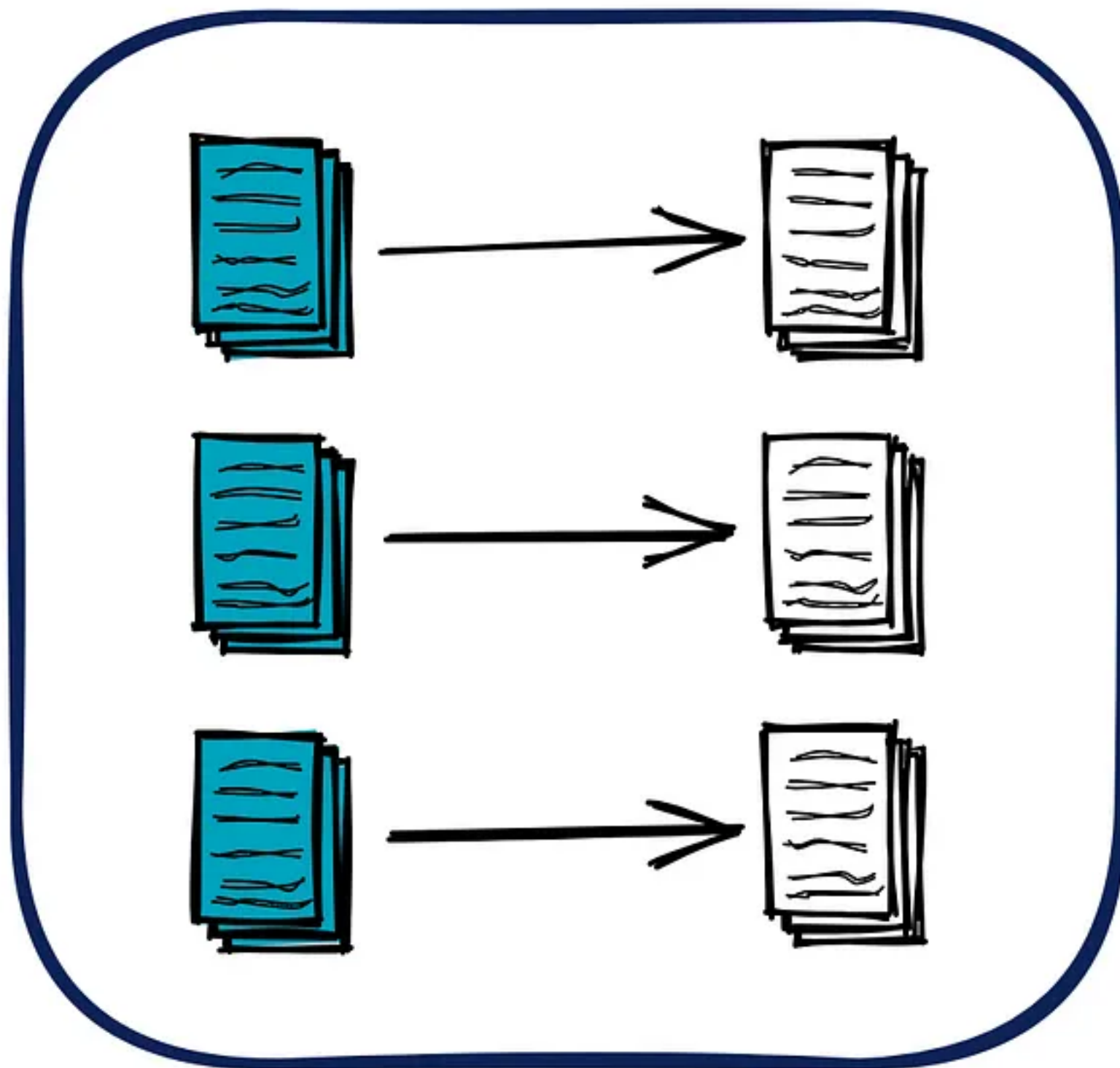
- ❖ transformations where each input partition **contributes to only one output partition.**
- ❖ Examples include **map, filter, and flatMap.**
- ❖ Narrow transformations can be **computed in parallel without shuffling data across partitions.**

❖ **Wide Transformations**

- ❖ transformations where each input partition may **contribute to multiple output partitions.**
- ❖ Examples include **groupBy, reduceByKey, and join.**
- ❖ Wide transformations require **shuffling data across partitions and may involve data movement between nodes.**

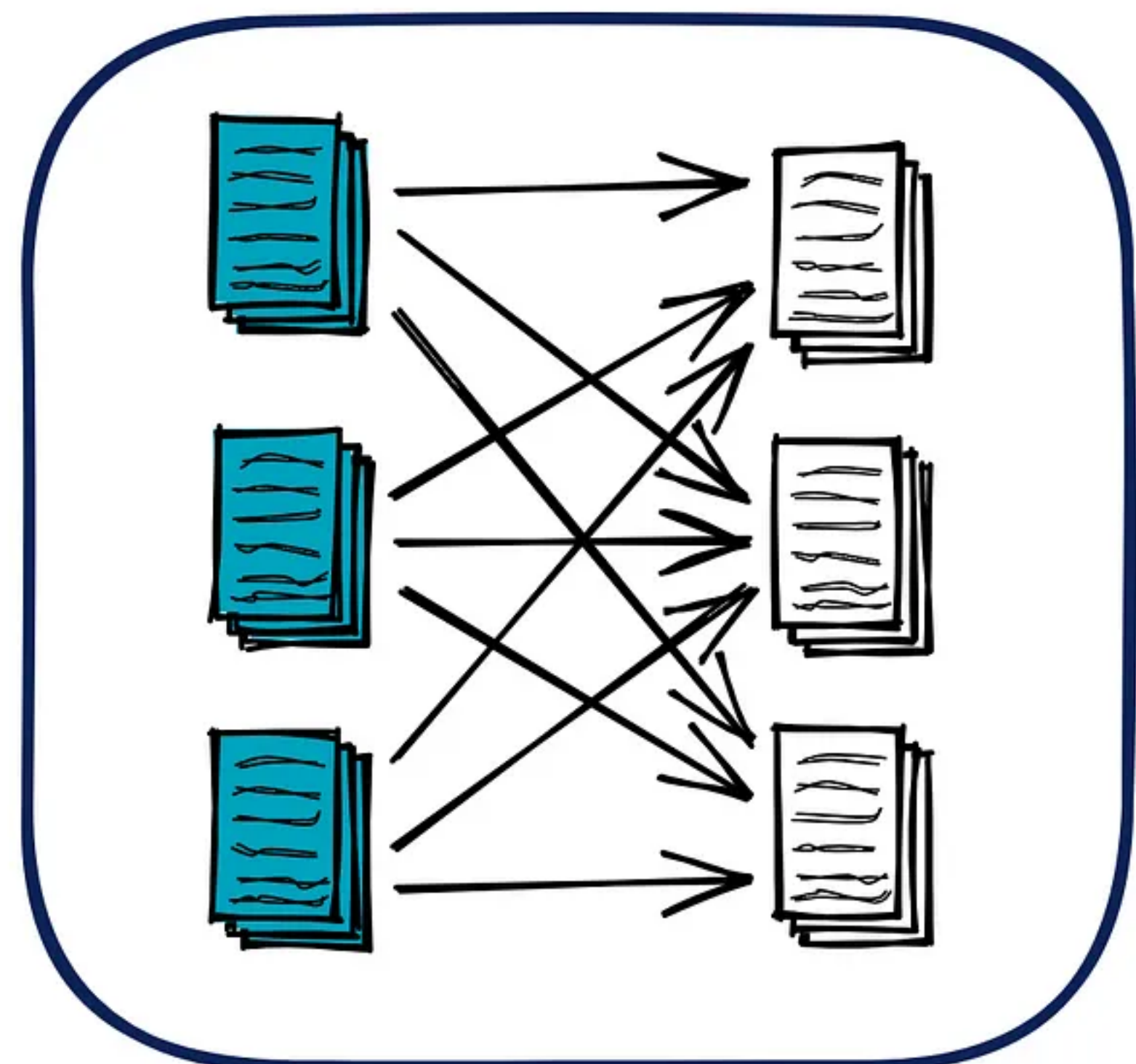
Narrow & Wide Transformations

Narrow Transformations



E.g. `map()`, `filter()`

Wide Transformations



E.g. `groupBy()`, `join()`

Partition

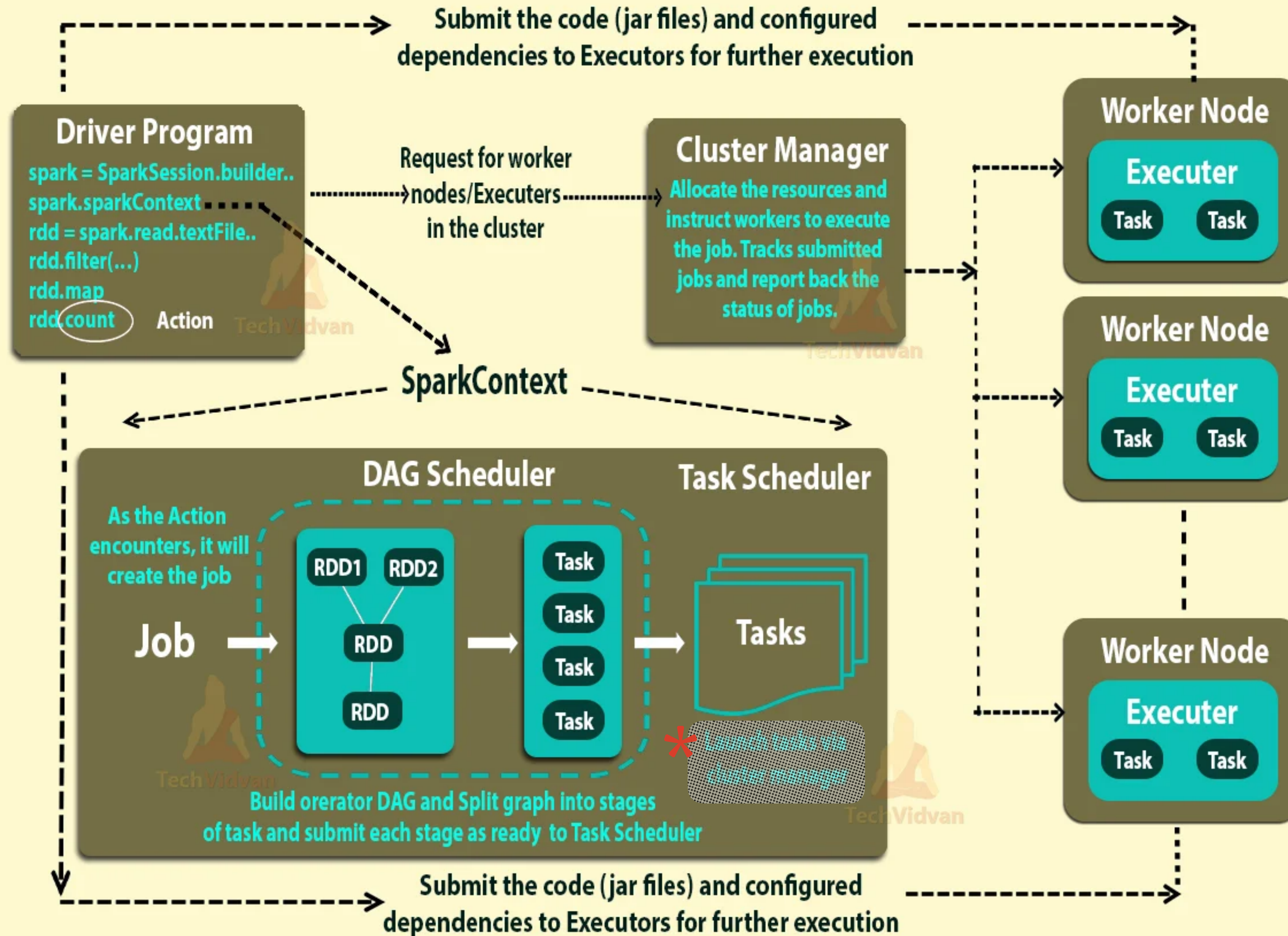
- ❖ A partition in Spark is a **smaller, logical division of data that allows computations to be performed in parallel.**
- ❖ Each partition is **processed by a single task in a single executor.**
- ❖ Spark **automatically determines the number of partitions** based on the size of the data and the available resources.

Shuffle in Spark

- ❖ Shuffling is **the process of distributing data across the cluster workers** in order to process it in parallel.
- ❖ It happens generally
 - ❖ when data is not evenly distributed, (data skewing)
 - ❖ when data should be arranged in a specific way to be processed
 - ❖ **when data needs to be aggregated or joined across partitions**, requiring data to be moved between nodes in the cluster.
 - ❖ when there is not enough memory on a single node to store all the required data for processing.
- ❖ Exchange in DAG represents Shuffling, i.e., physical data movement on the cluster among several nodes.

How Spark Works Internally

Internals of Job Execution In Spark

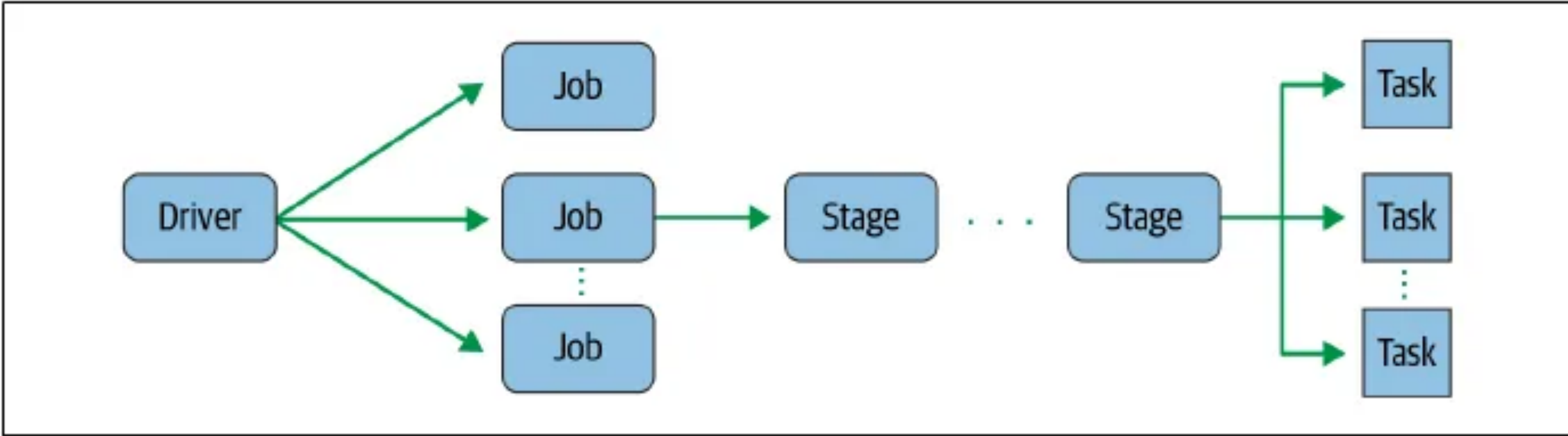


Executing Spark Jobs (1)

- ❖ Once you do a Spark submit, a driver program is launched and this requests for resources to the cluster manager and at the same time the main program of the user function of the user processing program is initiated by the driver program.
- ❖ Based on that, the execution logic is processed and parallelly Spark context is also created. Using the Spark context, the different transformations and actions are processed. So, **till the time the action is not encountered, all the transformations will go into the Spark context in the form of DAG that will create RDD lineage.**
- ❖ **Once the action is called job is created. Job is the collection of different task stages. Once these tasks are created, they are launched by the cluster manager on the worker nodes and this is done with the help of a class called task scheduler.**

Executing Spark Jobs (2)

- ❖ **The conversion of RDD lineage into tasks is done by the DAG scheduler.** Here DAG is created based on the different transformations in the program and once the action is called these are split into different stages of tasks and submitted to the task scheduler as tasks become ready.
- ❖ Then these are launched on the different executors in the worker node through the cluster manager. The entire resource allocation and the tracking of the jobs and tasks are performed by the cluster manager.
- ❖ As soon as you do a Spark submit, **your user program and other configuration mentioned are copied onto all the available nodes in the cluster.** So that the program becomes **the local read on all the worker nodes**. Hence, the parallel executors running on the different worker nodes do not have to do any kind of network routing.



Job

- ❖ A parallel computation consisting of multiple tasks that **gets spawned in response to a Spark action** (e.g., `save()`, `collect()`).
- ❖ During interactive sessions with Spark shells, the driver converts your Spark application into one or more Spark jobs.
- ❖ It then transforms each job into a DAG. This, in essence, is Spark's execution plan, where each node within a DAG could be a single or multiple Spark stages.
- ❖ So a spark job is **created whenever an action is called**.

Stage and Task

- ❖ **Stage** — Each job gets divided into **smaller sets of tasks called stages that depend on each other**. As part of the DAG nodes, stages are created **based on what operations can be performed serially or in parallel**. Not all Spark operations can happen in a single stage, so they may be divided into multiple stages. Often stages are delineated on the operator's **computation boundaries**, where they dictate data transfer among Spark executors.
- ❖ **Task** — A single unit of work or execution that will be sent to a Spark executor. Each stage is comprised of Spark tasks (a unit of execution), which are fed across each Spark executor, **each task maps to a single partition of data**.

Tasks and Stages in Spark (1)

- ❖ Task and Stage Formation:

- ❖ Tasks: Tasks in Spark are units of work that are executed on partitions of data.
- ❖ Stages: Stages are groups of tasks that can be executed together, **typically as a result of a narrow transformation (like map or filter) that does not require shuffling data across the cluster.**

- ❖ Boundary of Stages:

- ❖ A stage boundary is **typically defined by operations that require data to be shuffled across the cluster.**
- ❖ Examples include operations like **reduceByKey, groupByKey, join, or any operation that causes a data exchange (shuffle) between different partitions of data.**
- ❖ When such an operation is encountered in a Spark job, it marks the end of the current stage and the beginning of a new stage.

Tasks and Stages in Spark (2)

- ❖ Data Transfer During Stage Execution:
 - ❖ Within a stage, tasks operate on **data that is already partitioned across the cluster**. Tasks within the same stage can **operate independently on their respective partitions without needing to exchange data**.
 - ❖ When a stage completes and a shuffle boundary is encountered, **Spark reorganizes data across the cluster according to the requirements of the subsequent stage**.
 - ❖ Data transfer among Spark executors primarily occurs during these shuffle operations. **Executors exchange data partitions to ensure that each executor has the data it needs for the next stage of computation**.

Tasks and Stages in Spark (3)

- ❖ Impact on Performance:
 - ❖ Efficient stage management is crucial for Spark job performance. **Minimizing the number of stages and optimizing data partitioning** can reduce the overhead associated with data shuffling and improve overall execution time.
 - ❖ Understanding stage boundaries helps in optimizing Spark jobs by reducing unnecessary data movements and improving resource utilization.

Example Scenario

- ❖ Consider a simple Spark job that reads data, performs a map operation (forming one stage), then performs a reduceByKey operation (forming another stage due to shuffle).
- ❖ Between these stages, data will be shuffled across the cluster based on the keys used in the reduceByKey operation.


```
from pyspark.sql import SparkSession  
  
sc = SparkContext(conf=conf)  
  
rdd = sc.textFile("path_to_your_data.csv")  
  
rdd_filtered = rdd.filter(lambda x: x[2] > 0)  
  
rdd = rdd.map(parse_row).filter(lambda x: x is not None)  
  
total_trips = rdd_filtered.count()
```



```
from pyspark.sql import SparkSession

sc = SparkContext(conf=conf)

rdd = sc.textFile("path_to_your_data.csv")

rdd_filtered = rdd.filter(lambda x: x[2] > 0)

rdd = rdd.map(parse_row).filter(lambda x: x is not None)

total_trips = rdd_filtered.count()

all_records_count = rdd.count()
```


Why DAG?

What was the problem in MapReduce job execution?

- ❖ A MapReduce job is a **strict two-stage pipeline**: Map $\rightarrow$ Reduce.
- ❖ To do anything beyond that (e.g., Map $\rightarrow$ Reduce $\rightarrow$ Map $\rightarrow$ Reduce), you need chained jobs, which:
 - ❖ **Are hard to optimize globally.**
 - ❖ **Introduce unnecessary waits and disk I/O between jobs.**
 - ❖ **Have no visibility into overall data flow.**

Motivation for DAGs

- ❖ **Overcoming the One-Job-at-a-Time Constraint in MapReduce**

What did Spark do differently?

- ❖ Introduced **DAG (Directed Acyclic Graph) execution**.
- ❖ Spark **builds a logical DAG of stages** across multiple transformations before executing anything.
- ❖ Allows **global optimization** (e.g., pipelining narrow transformations, minimizing shuffles).
- ❖ **Key Benefit of DAGs:**
 - ❖ Smarter execution plans with fewer stages, less disk I/O, and better use of cluster resources.

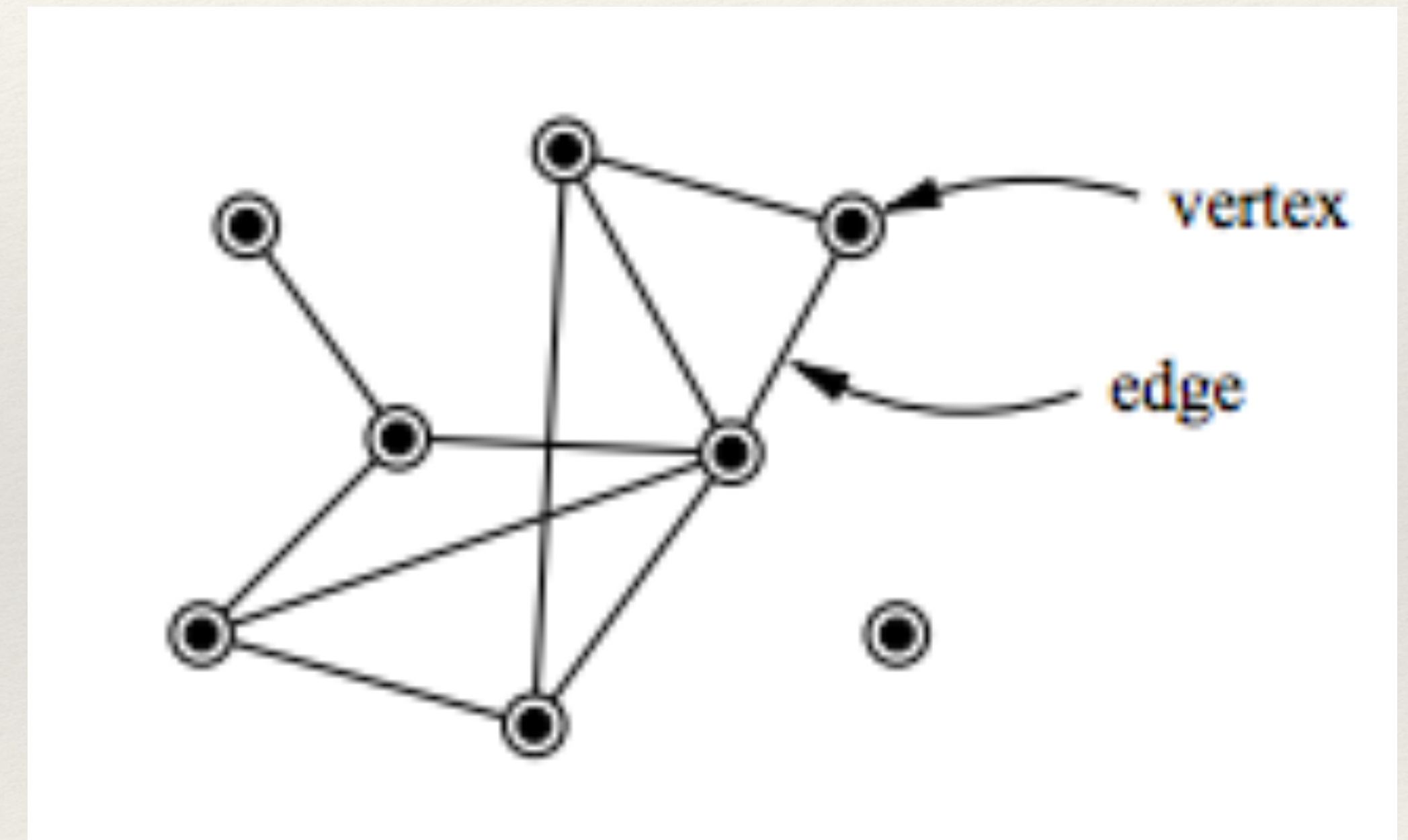
DAG

❖ Directed Acyclic Graph

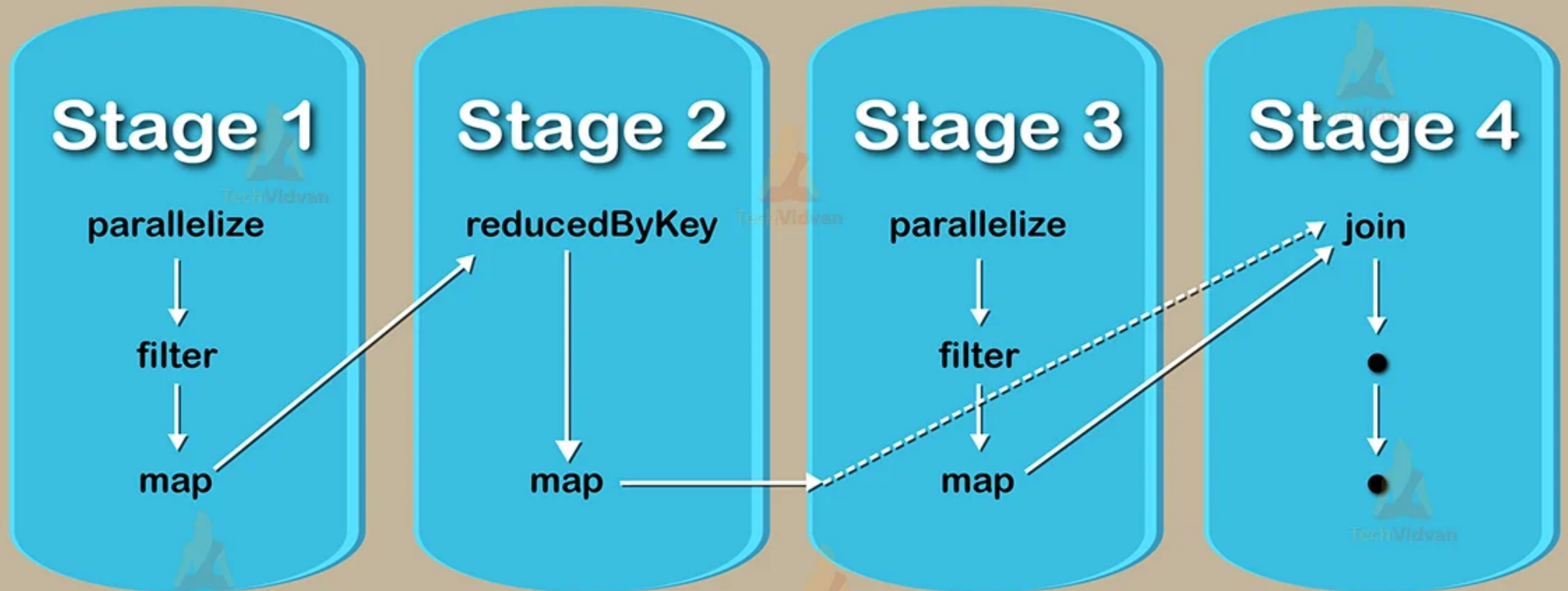
- ❖ Directed – here means that each edge has a defined direction, that represents **a unidirectional flow from one vertex to another**. This creates a sequence i.e. each node is in linkage from earlier to later in the appropriate sequence.
- ❖ Acyclic – here means that **there are no loops or cycles in the graph**, so that for any given vertex, if we follow an edge that connects that vertex to another, there is no path in the graph to get back to that initial vertex. Once a transformation takes place it cannot return to its earlier position.
- ❖ Graph – From graph theory, it is **a combination of vertices and edges**. Those pattern of connections together in a sequence is the graph.

Graph - Vertices and Edges

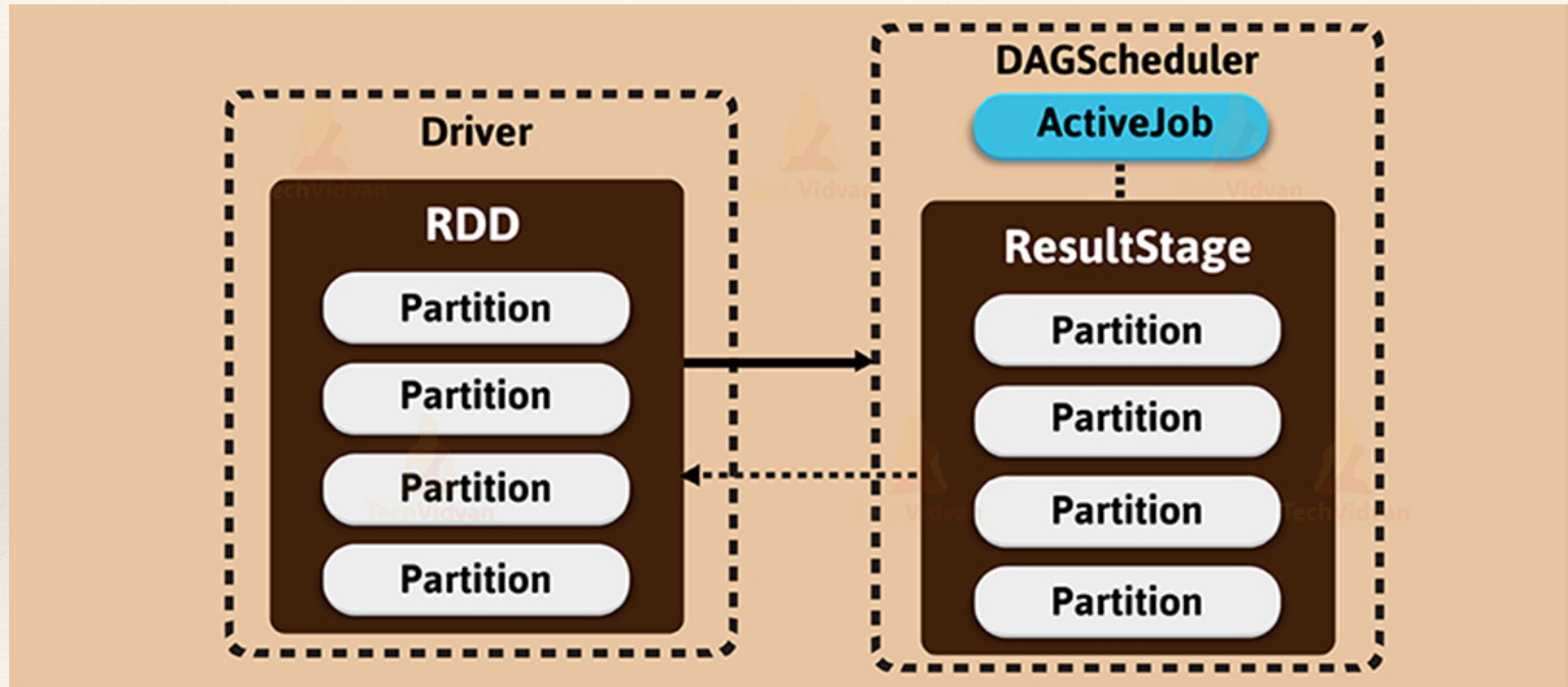
- ❖ The order of the activities is represented by a graph, which is visually presented as a set of circles, each circle represent an activity, connected by lines, which represent the execution flow from one activity to another. Circles are known as a **vertices** and connecting lines are known as **edges**.



DAG Visualisation



Executing Action Leads to New ResultStage and ActiveJob in DAGScheduler



Why Spark Introduced RDDs and DAGs

Feature	MapReduce Limitation	Spark Solution	Result
Data Reuse	No reuse; data must be recomputed	RDDs with caching	Faster iterative workloads
Fault Tolerance	Replication-heavy, slow recovery	RDD lineage	Lightweight, efficient recovery
Execution Model	Fixed Map→Reduce pipeline	DAG-based execution	Flexible, optimized workflows
Multi-Stage Jobs	Manual chaining, inefficient	Global DAG planning	Fewer I/Os, better performance

RDD vs DataFrames

RDD vs DataFrames

- ❖ While an RDD is the basic data structure of Spark, it is a lower-level API that requires a more verbose syntax and lacks the optimizations provided by higher-level data structures.
- ❖ Spark shifted toward a more user-friendly and optimized API with the introduction of DataFrames—**higher-level abstractions built on top of RDDs. The data in a DataFrame is organized into named columns, structuring it more like the data in a relational database.**
- ❖ DataFrame operations also benefit from **Catalyst**, Spark SQL's optimized execution engine, which can increase computational efficiency, potentially improving performance.
- ❖ **Transformations and actions can be run on DataFrames the way they are in RDDs.**

RDD vs DataFrames

- ❖ Because of their higher-level API and optimizations, DataFrames are typically easier to use and offer better performance;
- ❖ however, due to their lower-level nature, RDDs can still be useful for defining custom operations, as well as debugging complex data processing tasks. RDDs offer more granular control over partitioning and memory usage.
- ❖ **When dealing with raw, unstructured data, such as text streams, binary files, or custom formats, RDDs can be more flexible, allowing for custom parsing and manipulation in the absence of a predefined structure.**

W5M2

- ❖ Documentation:
 - ❖ Provide a report explaining the DAG created, the steps of transformations and actions, and how Spark's internal mechanisms (lazy evaluation, stage optimization) were utilized.